Mini Project — Lecture 1: Foundations of LLMs

การฝึก MiniGPT สำหรับ Sequence Reversal
และการ Fine-tune ด้วย Reinforcement Learning (GRPO)

6733141121 Pawaris Klampol · 6733205721 Phoovich Phuengphaendin
6733168121 Patcharada Tawaditap · 6733148621 Panisa Taratchon
Computer Engineering and Digital Technology (CEDT) · 26 เมษายน 2568

บทนำ: รายงานนี้อธิบายการออกแบบและผลการทดลองของ Mini Project ซึ่งประกอบด้วยสองขั้นตอนหลัก ได้แก่ (1) การฝึก Transformer Decoder ขนาดเล็ก (MiniGPT) ตั้งแต่ต้นให้เรียนรู้การกลับลำดับตัวอักษร (sequence reversal) ด้วย Supervised Fine-Tuning (SFT) และ (2) การ fine-tune โมเดลด้วย Reinforcement Learning โดยใช้อัลกอริทึม GRPO เพื่อสอนให้โมเดลตัดตัวอักษร “b” ออกจาก output — ซึ่งเป็น constraint ที่ระบุตรง ๆ ผ่าน supervised loss ได้ยาก

1. การกำหนดค่า GPT Model Configuration

1.1. สถาปัตยกรรมโดยรวม

โมเดลที่ใช้มีชื่อว่า **MiniGPT** ซึ่งเป็น Transformer Decoder แบบ autoregressive แม้ว่าจะใช้ `nn.TransformerEncoder` ของ PyTorch ภายในก็ตาม โดยหลักการทำงานของ Decoder ถูกจำลองด้วยการใส่ **causal mask** แบบ upper-triangular matrix (สร้างด้วย `torch.triu`) ใน attention layer ทำให้ token ที่ตำแหน่ง t สามารถ attend ได้เฉพาะ token ที่ตำแหน่ง $t' \leq t$ เท่านั้น ป้องกันการ “ดู” token ในอนาคต

โมเดลประกอบด้วยส่วนหลัก 3 ส่วน:

1. Token Embedding Layer: แปลง token ID (integer) เป็น dense vector ขนาด `d_model` โดยใช้ `nn.Embedding(vocab_size, d_model)` ซึ่งมี weight ที่เรียนรู้ได้จำนวน $30 \times 256 = 7,680$ parameters

2. Sinusoidal Positional Encoding: เพิ่มข้อมูลตำแหน่งของ token แต่ละตัวในลำดับด้วยสูตร fixed (ไม่มี learnable parameters):

$$\mathrm{PE}(\mathrm{pos}, 2i) = \sin\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right) \quad \mathrm{PE}(\mathrm{pos}, 2i+1) = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$

3. TransformerEncoderLayer Stack + LM Head: ซ้อน N ชั้นของ TransformerEncoderLayer ซึ่งแต่ละชั้นประกอบด้วย Multi-Head Self-Attention, Feed-Forward Network, และ Layer Normalization ตามด้วย Linear layer สำหรับทำนาย logit ของ next token ทั้งหมด 30 ตัว

1.2. Hyperparameter ที่ใช้

Hyperparameter	ค่าที่ใช้	คำอธิบาย
`d_model`	256	ขนาดของ embedding / hidden dimension ทุก layer
`nhead`	8	จำนวน attention heads (ขนาดแต่ละ head = 256/8 = 32)
`num_layers`	4	จำนวนชั้นของ TransformerEncoderLayer
`dim_ff`	512	ขนาด hidden layer ใน Position-wise FFN
`dropout`	0.1	อัตรา dropout ใช้ระหว่าง training
`max_len`	256	ความยาวสูงสุดของ sequence ที่ Positional Encoding รองรับ
`vocab_size`	30	ขนาด vocabulary ทั้งหมด
จำนวน parameters รวม	2.1M	คำนวณจาก embedding + 4 layers + LM head

ตารางที่ 1: Hyperparameter ของ MiniGPT

หมายเหตุ: ค่า `d_model=256` และ `dim_ff=512` ที่ใช้นี้ใหญ่กว่าค่าที่แนะนำใน slide (D_MODEL=128, D_FF=128) ซึ่งช่วยให้โมเดลมี capacity สูงขึ้น ทำให้ train ได้ดีขึ้นบน task นี้แม้ใช้เวลานานกว่าเล็กน้อย

1.3. Vocabulary

Vocabulary มีขนาดคงที่ 30 token แบ่งเป็น:

- **Special tokens** (4 token): `<PAD>` (ID 0), `<BOS>` (ID 1), `<SEP>` (ID 2), `<EOS>` (ID 3)
- **ตัวอักษรภาษาอังกฤษพิมพ์เล็ก** (26 token): `a–z` (ID 4–29)

2. การเตรียม Dataset สำหรับ Training และ Evaluation

2.1. SFT Dataset: รูปแบบและการสร้าง

Dataset สำหรับขั้นตอน SFT ถูก generate แบบ on-the-fly โดย class `ReverseSequenceDataset` ซึ่ง inherit จาก `torch.utils.data.Dataset` ในแต่ละ `__getitem__` จะสุ่มสร้าง sequence ตัวอักษรภาษาอังกฤษพิมพ์เล็กความยาว 2–10 ตัวอักษร แล้วสร้าง token sequence ในรูปแบบ:

$$\underbrace{[\text{<BOS>} \quad x_1 \quad x_2 \quad \ldots \quad x_n \quad <SEP> \quad x_n \quad \ldots \quad x_2]}_{\text{input (x)}}$$

$$\underbrace{[x_1 \quad x_2 \quad \ldots \quad <SEP> \quad x_n \quad \ldots \quad x_1 \quad <EOS>]}_{\text{target (y)}}$$

โดย input และ target เป็น sequence เดียวกันที่เลื่อนออกไป 1 token (shift-by-one) ซึ่งเป็น paradigm มาตรฐานของการฝึก autoregressive language model

การ **Batching** ใช้ `collate_fn` แบบ dynamic padding กล่าวคือ pad ทุก sequence ใน batch ด้วย `<PAD>` ให้มีความยาวเท่ากับ sequence ที่ยาวที่สุดใน batch นั้น และส่ง `pad_mask` เข้า Transformer เพื่อให้ attention ข้าม PAD token

Split	จำนวน Samples	การใช้งาน
Train	50,000	ฝึกโมเดลด้วย gradient descent ทุก iteration
Validation	5,000	วัด val_loss เพื่อตัดสินใจ save checkpoint
Test	5,000	ประเมินผลสุดท้ายหลัง training จบ

ตารางที่ 2: การแบ่ง Dataset สำหรับ SFT — samples ทั้งหมด generate แบบ random ทุก epoch

2.2. RL Dataset: การ Sample แบบ Mixed

ใน RL stage ไม่มี pre-built dataset แต่ใช้ฟังก์ชัน `sample_seq_mixed()` สุ่มแบบ on-the-fly ซึ่งมีพฤติกรรมพิเศษเพื่อให้โมเดลได้เจอ skip-b case บ่อยครั้ง:

- **70% ของ sequence** (prob_has_b = 0.7): บังคับใส่ตัว `b` จำนวน 1–2 ตัว ในตำแหน่งที่สุ่ม
- **30% ของ sequence**: สุ่มตัวอักษรทั่วไปโดยไม่มีการบังคับ (อาจมีหรือไม่มี `b` ก็ได้)

ความยาว sequence ใน RL อยู่ที่ 2–6 ตัวอักษร (สั้นกว่า SFT range ที่ 2–10) เนื่องจากต้องการให้การ training เร็วขึ้นและมี curriculum learning ที่ควบคุมได้ง่ายกว่า

2.3. Fixed Test Sets สำหรับ Evaluation

เพื่อให้การประเมินผลของทุก checkpoint สามารถเปรียบเทียบกันได้อย่าง fair จึงสร้าง fixed test set ด้วย random seed คงที่ (seed=42):

ชุด Test	จำนวน Samples	Length Range	Seed
In-distribution	500	2–6	42
Out-of-distribution (OOD)	500	7–15	42
Per-length breakdown	100 samples/length	2–15	42

ตารางที่ 3: Fixed Test Sets ที่ใช้ประเมินผล

Metrics หลัก:

- **`exact_match`**: สัดส่วน output ที่ตรงกับ skip-b target ทุก token (strict)
- **`no_b_rate`**: สัดส่วน output ที่ไม่มีตัว `b` เลย (วัด skip behavior อย่างเดียว)

3. ประสิทธิภาพของ Pretrained SFT Model

3.1. กระบวนการฝึก SFT

SFT model ถูกฝึกด้วย optimizer **AdamW** (learning rate = 3×10^{-4}, weight_decay default) เป็นเวลา **40 epochs** บน training set ขนาด 50,000 samples (batch_size=64) โดยใช้ **gradient clipping** (max_norm=1.0) และ **ReduceLROnPlateau scheduler** (factor=0.5, patience=2) ที่จะลด learning rate ลงครึ่งหนึ่งเมื่อ val_loss ไม่ดีขึ้นต่อเนื่อง 2 epoch Loss function คือ cross-entropy โดย ignore `<PAD>` token

Checkpoint ถูก save เมื่อ val_loss ต่ำที่สุด ซึ่งจาก training log พบว่าโมเดลสามารถเรียนรู้ task reversal ได้ดีภายใน 10–15 epoch แรก และ val_loss stabilize ในช่วงต่อมา

3.2. ผลการประเมิน SFT Model

เนื่องจากขั้นตอน SFT ไม่ได้ฝึกให้ skip ตัว `b` ผลการประเมินบน fixed test set (ที่วัดเทียบ **skip-b target**) จึงสะท้อนว่าโมเดลได้คะแนนเฉพาะ sequence ที่ไม่มีตัว `b` อยู่เดิมและ reverse ถูกต้อง (30% ของ test set ไม่มี `b`)

ชุด Evaluation	exact_match	no_b_rate
In-distribution (len 2–6, n=500)	22.80%	22.80%
Out-of-distribution (len 7–15, n=500)	9.40%	22.40%

ตารางที่ 4: ประสิทธิภาพ SFT Model เทียบกับ skip-b target (n=500, seed=42)

ตัวอย่างจาก sanity check ยืนยันว่า SFT สามารถ **reverse ได้ถูกต้อง** แต่ยังไม่รู้จัก skip constraint:

- `tesbt` → SFT ทำนาย `tbset` (reverse ถูกต้องแต่มี `b` ใน output)
- `abcde` → SFT ทำนาย `edcba` (reverse ถูกต้องแต่มี `b` ใน output)
- `qwer` → SFT ทำนาย `rewq` (ถูกต้องทุกด้านเพราะไม่มี `b`)

ความแตกต่างระหว่าง in-dist (22.80%) และ OOD (9.40%) สะท้อนว่า SFT model มีปัญหาการ generalize ไปยัง sequence ที่ยาวกว่า training range ซึ่งเป็นลักษณะปกติของ Transformer ที่ไม่ได้ฝึกบน positional encoding ที่ยาวกว่า

4. การออกแบบ RL Training และ Reward Function

4.1. อัลกอริทึม GRPO

การ fine-tune ใช้ **GRPO (Group Relative Policy Optimization)** ซึ่งเป็น variant ของ policy gradient ที่ออกแบบมาสำหรับ language model โดยเฉพาะ ข้อแตกต่างหลักจาก PPO คือ GRPO **ไม่ต้องการ value network แยกต่างหาก** แต่ใช้ reward ภายใน group ของ rollout เดียวกันเป็น baseline แทน

ขั้นตอนใน 1 training step:

1. Sample batch ของ input sequence (batch_size = 8)

2. สำหรับแต่ละ sequence ทำ stochastic rollout จำนวน `grpo_g = 4` ครั้ง โดยแต่ละครั้ง sample token จาก softmax distribution (ไม่ใช่ greedy)
3. คำนวณ reward r_1, r_2, r_3, r_4 สำหรับแต่ละ rollout ด้วย `compute_reward()`
4. คำนวณ **group-normalized advantage**: $A_i = (r_i - \bar{r}_{\text{group}})/(\sigma_{\text{group}} + \varepsilon)$ โดย $\varepsilon = 10^{\{-8\}}$
5. คำนวณ total loss: $\mathcal{L} = \underbrace{-\frac{1}{N}\sum_i \log \pi_\theta(a_i \mid s_i) \cdot A_i}_{\text{policy loss}} + \underbrace{\lambda_{\text{KL}} \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})}_{\text{KL penalty}} - \underbrace{\lambda_H \cdot H(\pi_\theta)}_{\text{entropy bonus}}$
6. Update model parameters ด้วย AdamW + gradient clipping

4.2. Reward Function

Reward function (`compute_reward`) ออกแบบให้ครอบคลุม behavior ที่ต้องการอย่างรอบด้าน:

เงื่อนไข	Reward	เหตุผลในการออกแบบ
Exact match กับ skip-b target	+5.0	Signal หลักที่แรงที่สุด — reward เต็มเมื่อถูกต้องสมบูรณ์
Positional match (ต่อ position)	+0.2 × (ถูก/total)	Dense signal: ให้ credit แม้ผิดบางตำแหน่ง
Character coverage (multiset)	+0.1 × (overlap/total)	ให้ credit กับ token ที่มีใน output แม้ผิดตำแหน่ง
มีตัว `b` ใน output	−1.0 × count	Penalty หลักสำหรับ skip-b objective
ความยาวต่างจาก target	−0.1 × |Δlen|	บังคับให้ output มีความยาวถูกต้อง
ไม่มี `<EOS>` ใน output	−0.5	บังคับให้ generate ครบจนถึง end token
มี `<PAD>` ใน output	−2.0 × count	ไม่ควร generate padding token ออกมา
มี `<SEP>` หรือ `<BOS>` ใน output	−2.0 × count	ไม่ควร generate special token ผิดที่

ตารางที่ 5: องค์ประกอบทั้งหมดของ Reward Function

4.3. เทคนิคเสริมใน RL Training

KL Penalty (`kl_coef = 0.2`): คำนวณ KL divergence ระหว่าง policy model กับ frozen SFT reference model บน prompt tokens แล้วนำมาบวกใน loss เพื่อป้องกัน model เบี่ยง

ออกจาก SFT มากเกินไป (catastrophic forgetting) Reference model ถูก freeze ตั้งแต่เริ่ม RL และไม่มีการ update gradient

Entropy Bonus (`entropy_coef = 0.005`): เพิ่ม negative entropy เป็น bonus เพื่อส่งเสริมการ explore ป้องกัน policy collapse ไปยัง deterministic output เดิม

Curriculum Learning: `cur_max_len` เริ่มที่ 3 (= min_len + 1) แล้วเพิ่มทีละ 1 ทุก `curriculum_steps = 3,000` steps จนถึง `max_len = 6` ทำให้โมเดลเรียนรู้จาก case ง่ายก่อนแล้วค่อยเพิ่ม complexity

Cosine LR Scheduler: `CosineAnnealingLR` (T_max = 15,000 steps, eta_min = 3×10^{-6}) decay learning rate จาก 3×10^{-5} ลงอย่างราบรื่น ป้องกัน oscillation ในช่วง late training

5. ประสิทธิภาพของ Fine-tuned RL Model

RL model บันทึก checkpoint ที่ **step 9,400** ด้วย best_exact_match = **100.00%**

5.1. ผล Sanity Check

Input	SFT Output	RL Output	Target (skip-b)	RL ถูก?
tesbt	tbset	tset	tset	✓
abcde	edcba	edca	edca	✓
bomb	bmob	mo	mo	✓
robot	tobor	toor	toor	✓
bbba	abbb	a	a	✓
asdfb	bfdsa	fdsa	fdsa	✓
qwer	rewq	rewq	rewq	✓
banana	ananab	anana	anana	✓

ตารางที่ 6: ผล Sanity Check เปรียบเทียบ SFT กับ RL model

5.2. In-distribution และ OOD Evaluation

ชุด Evaluation	SFT exact	RL exact	SFT no_b	RL no_b
In-dist (len 2–6, n=500)	22.80%	**100.00%**	22.80%	**100.00%**
OOD (len 7–15, n=500)	9.40%	14.40%	22.40%	69.00%

ตารางที่ 7: ผลการประเมินบน Fixed Test Set (seed=42)

5.3. Per-length Breakdown

Length	SFT exact	RL exact	Delta	RL no_b
2	32.00%	100.00%	+68.00%	100.00%
3	34.00%	100.00%	+66.00%	100.00%
4	30.00%	99.00%	+69.00%	100.00%
5	21.00%	100.00%	+79.00%	100.00%
6	25.00%	98.00%	+73.00%	100.00%
7	13.00%	40.00%	+27.00%	66.00%
8	24.00%	41.00%	+17.00%	51.00%
9	20.00%	33.00%	+13.00%	43.00%
10	17.00%	23.00%	+6.00%	44.00%
11	0.00%	5.00%	+5.00%	54.00%
12–15	0.00%	0.00%	0.00%	56–100%

ตารางที่ 8: Per-length Breakdown (100 samples/length, seed=42) — สีเขียวอ่อนคือ in-distribution range (len 2–6)

5.4. Edge Cases

กรณี	Input	RL Output	Target	ถูก?
ทุกตัวเป็น b	`bbbbb`	(ว่าง)	(ว่าง)	✓
ไม่มีตัว b เลย	`qwerty`	`ytrewq`	`ytrewq`	✓
ตัวอักษรตัวเดียว	`a`	`a`	`a`	✓
b ตัวเดียว	`b`	`j`	(ว่าง)	✗
ยาวมาก + มี b	`basketball`	`llabteksa`	`llateksa`	✗
ทุกตัวเหมือนกัน	`aaaaa`	`aaaaa`	`aaaaa`	✓

ตารางที่ 9: ผล Edge Case Testing บน RL Model

6. การวิเคราะห์และอภิปรายผลการทดลอง

6.1. ความสำเร็จของ RL Fine-tuning

ผลการทดลองแสดงให้เห็นอย่างชัดเจนว่า GRPO สามารถสอน MiniGPT ให้เรียนรู้ constraint ใหม่ที่ไม่ได้มีใน SFT objective ได้อย่างมีประสิทธิภาพสูง โดยบรรลุ **100% exact_match และ 100% no_b_rate** บน in-distribution test set (length 2–6) ที่ step 9,400 จากทั้งหมด 15,000 steps ซึ่งเร็วกว่าที่คาดไว้ แสดงให้เห็นว่า SFT base model มีพื้นฐานที่ดีพอที่จะ fine-tune ด้วย RL ได้อย่างรวดเร็ว

ผลนี้ยังตอกย้ำว่า reward function ที่ออกแบบมานั้นมี signal ที่เพียงพอและสมดุล โดยเฉพาะการให้ dense reward ผ่าน positional match และ character coverage ช่วยให้ policy gradient มี variance ต่ำลงเมื่อเทียบกับการให้ sparse reward เฉพาะ exact match เพียงอย่างเดียว

6.2. Generalization Gap: In-distribution vs OOD

ปัญหา generalization ไปยัง sequence ที่ยาวกว่า training range เป็นข้อจำกัดหลักของโมเดลนี้ โดยสรุปได้ดังนี้:

- **Exact_match** ลดลงจาก 99% (len 2–6) → 40% (len 7) → 0% (len 12+) อย่างรวดเร็ว
- **No_b_rate** ลดลงช้ากว่ามาก: ยังคงสูงถึง 54–100% สำหรับ len 11–15

ข้อสังเกตสำคัญคือ **no_b_rate generalize ได้ดีกว่า exact_match อย่างมีนัยสำคัญ** หมายความว่าโมเดลเรียนรู้ skip-b behavior ในระดับที่ transfer ไปยัง sequence ยาวได้ แต่ reversal accuracy ยังคงถูกจำกัดโดย training distribution เนื่องจาก positional encoding ไม่ได้ถูกฝึกบน context ที่ยาวพอ

6.3. วิเคราะห์ Edge Cases

กรณีที่น่าสนใจคือ `b` ตัวเดียว (input = `"b"`) ที่โมเดล predict เป็น `j` แทน empty sequence มีสาเหตุที่เป็นไปได้ดังนี้:

1. ใน training set sequences ที่เป็น `b` ล้วนพบน้อยมาก เพราะ `min_len=2` ทำให้ sequence ยาว 1 ตัวไม่ถูกสุ่ม (จริงๆ single-char ก็ไม่ควรปรากฏใน training)
2. โมเดลมี prior จาก SFT ว่าต้อง generate ตัวอักษรอย่างน้อย 1 ตัวก่อน `<EOS>`
3. Length penalty (-0.1×|Δlen|) อาจไม่แรงพอที่จะ overcome prior นี้

กรณี `basketball` ที่ยาวเกิน training range แสดง pattern การ reverse บางส่วนแต่ไม่สมบูรณ์ ซึ่งสอดคล้องกับ per-length breakdown ที่ len=10 ยังมี exact_match 23%

6.4. ผลของ KL Penalty ต่อ Catastrophic Forgetting

หลักฐานที่แสดงว่า KL penalty ทำงานได้ดีคือ RL model ยังสามารถ reverse sequence ที่ไม่มี `b` ได้อย่างถูกต้อง (เช่น `qwer` → `rewq`) ซึ่งเป็นความสามารถที่ได้มาจาก SFT หาก KL penalty ต่ำเกินไปหรือไม่มีเลย โมเดลมีความเสี่ยงที่จะ collapse ไปยัง strategy ง่าย เช่น generate `<EOS>` ทันทีเพื่อหนี `b` penalty

6.5. สรุปและแนวทางพัฒนา

โปรเจกต์นี้แสดงให้เห็นว่า (1) Transformer Decoder ขนาดเล็ก 2.1M parameters สามารถเรียนรู้ structured task ได้อย่างมีประสิทธิภาพ และ (2) GRPO เป็นอัลกอริทึมที่ใช้งานได้จริงสำหรับการสอน behavior ที่ไม่สามารถระบุได้ง่ายผ่าน supervised learning

แนวทางปรับปรุงที่น่าสนใจ ได้แก่:

- ขยาย RL training range ให้ครอบคลุม length 2–12 หรือมากกว่า เพื่อปรับปรุง OOD generalization
- เพิ่ม `grpo_g` หรือ `batch_size` เพื่อลด variance ของ gradient

- ทดลองใช้ reward shaping เพิ่มเติม เช่น bonus เมื่อ output ว่างและ input เป็น b ล้วน

การเปิดเผยการใช้ AI (AI Disclosure): *รายงานนี้จัดทำโดยใช้ Claude Code (claude.ai/code) ช่วยในการ review โค้ด ตรวจสอบความถูกต้องของ implementation และช่วย generate โครงสร้างและเนื้อหาของรายงาน ผู้เขียนได้ตรวจสอบ เข้าใจ และรับผิดชอบเนื้อหาทุกส่วนที่ปรากฏในรายงานนี้*